
ISTEC Porto
Instituto Superior de Tecnologias Avançadas do Porto
Licenciatura em Engenharia Informática
Ano Letivo: 2025/2026

Unidade Curricular: Programação II
Proposta de Projeto Prático de Programação II

Autor:

Gonçalo Silva Pereira
goncalo.pereira.2022051@my.istec.pt

Estudante do 1º ano da Licenciatura de Engenharia Informática

Docente: João Rebelo

Porto, 13 junho de 2026

1. Introdução

O presente documento constitui a proposta oficial para o desenvolvimento do projeto prático da unidade curricular de Programação II. O projeto consiste no desenvolvimento do **DeskFlow**, um sistema desktop de HelpDesk e suporte técnico para clientes, enquadrado nas sugestões fornecidas no enunciado da UC. A aplicação será integralmente desenvolvida em C#, recorrendo à tecnologia Windows Presentation Foundation (WPF) com XAML para a criação de uma interface gráfica interativa e reativa. A lógica do sistema assentará nos pilares da Programação Orientada a Objetos (POO) e a persistência de dados será realizada através de uma ligação a uma base de dados relacional SQL Server.

2. Descrição do Problema

As empresas modernas enfrentam grandes dificuldades em gerir os pedidos de assistência, dúvidas e incidentes reportados pelos seus clientes. Os principais problemas identificados na ausência de um sistema estruturado são:

- **Perda de rastreabilidade:** Pedidos feitos por e-mail ou telefone perdem-se facilmente, impossibilitando o controlo de quais os problemas que já foram resolvidos e quais estão pendentes;
- **Falta de priorização:** Sem uma triagem automatizada, incidentes críticos (ex: falha total de um serviço) são tratados com a mesma urgência que dúvidas simples, prejudicando o nível de serviço;
- **Inexistência de histórico:** A dificuldade em associar um histórico de incidentes para um cliente específico impede que os técnicos identifiquem problemas recorrentes na infraestrutura ou nos produtos do cliente.

3. Enumeração dos Objetivos

O objetivo central deste projeto individual é consolidar de forma prática os conteúdos programáticos da UC, desenvolvendo uma solução comercialmente realista e totalmente funcional. Os objetivos específicos incluem:

- **Modelagem de Dados em POO:** Desenhar uma estrutura de classes limpa, com métodos construtores, encapsulamento estrito (atributos privados e propriedades públicas com *getters/setters*);
- **Implementação de Relações UML:** Garantir a aplicação prática de, pelo menos, 3 tipos de relações entre as classes da solução (Herança, Associação e Composição);
- **Persistência com Stored Procedures:** Ligar a aplicação ao SQL Server, garantindo que todas as operações de CRUD (criar clientes, abrir tickets, atualizar estados) sejam executadas estritamente através da invocação de *Stored Procedures*;
- **Desenvolvimento de UI em WPF:** Criar uma interface amigável utilizando controlos comuns (como ListBox para a fila de tickets, ComboBox para prioridades e DataGrid para dados dos clientes), com tratamento de erros robusto (*try-catch*) e validação de dados de entrada.

4. Estado da Arte

No mercado atual existem ferramentas de HelpDesk de grande escala (como o **Zendesk** ou **Jira Service Desk**). No entanto, estas plataformas são complexas, exigem subscrições dispendiosas na nuvem e requerem formação avançada para os utilizadores. Por outro lado, soluções baseadas em registos em papel ou folhas de cálculo pecam pela falta de integridade relacional e controlo de acessos. O projeto **DeskFlow** posiciona-se como uma aplicação desktop dedicada, focada na rapidez de resposta do técnico, que simplifica o fluxo de triagem sem sobrecarregar o operador com funcionalidades redundantes.

5. Metodologia

A solução será desenvolvida seguindo uma arquitetura modular em camadas autónomas para assegurar a manutenibilidade do código:

- **Camada de Apresentação (WPF/XAML):** Responsável pela interação visual com o utilizador técnico, tratamento de eventos (cliques em botões, seleção de linhas) e apresentação visual dos dados;
- **Camada de Lógica de Negócio (POO em C#):** Camada central onde são aplicadas as regras de negócio do HelpDesk (ex: calcular tempo de resolução, alterar o estado de um ticket de "Aberto" para "Resolvido");
- **Camada de Persistência (SQL Server):** Mapeamento e gravação dos objetos em tabelas relacionais através de um canal ADO.NET, delegando a lógica de inserção e atualização para as *Stored Procedures* na base de dados.

6. Desenvolvimento e como vão apurar os resultados

O desenvolvimento será dividido nos seguintes módulos funcionais principais:

- **Módulo de Gestão de Clientes:** Registo e listagem de clientes com validação de dados (impedir emails inválidos ou campos vazios);
- **Módulo de Gestão de Tickets:** Criação, atribuição e fecho de chamados técnicos de suporte;
- **Rotinas de Detecção de Erros:** Implementação de mecanismos de validação para capturar falhas de ligação à base de dados ou a inserção de dados incompatíveis, garantindo que a aplicação não crashe inesperadamente;
- **Apuramento de Resultados:** A aplicação incluirá uma área de relatório/dashboard simples (conforme pedido no ponto 2 do enunciado). O apuramento da eficácia do sistema será feito através da extração de métricas (ex: total de tickets abertos vs. resolvidos, e tempo médio de resolução). A coerência dos dados será apurada cruzando os contadores dos objetos gerados em C# com os resultados de funções agregadoras executadas no SQL Server.

7. Abordagem do Modelo de Classes

Para cumprir rigorosamente o requisito mínimo de apresentar **pelo menos 3 tipos de relações entre as classes** com propriedades completas, a arquitetura do **DeskFlow** foi estruturada da seguinte forma:

1. Relação de Herança (Especialização):

- Existirá uma classe abstrata chamada Pessoa (com os atributos privados id, nome e contacto, expostos por propriedades públicas);
- As classes Cliente (adiciona numeroContribuinte) e Tecnico (adiciona especialidade) irão herdar diretamente de Pessoa, demonstrando o conceito de especialização e herança.

2. Relação de Composição:

- A classe Ticket terá uma relação de **composição** com uma classe interna chamada HistoricoIntervencao (cardinalidade 1 -> 0..*). Se um ticket for apagado do sistema, todo o histórico de notas e intervenções técnicas associado a esse ticket deixa de fazer sentido e é eliminado em cascata.

3. Relação de Associação:

- Existirá uma **associação simples** entre a classe Ticket e as classes Cliente (quem abriu o chamado) e Técnico (quem vai resolver o chamado). Isto permite interligar os objetos de forma limpa na lógica do C# e refletir fielmente as chaves estrangeiras que serão criadas no SQL Server.